LDAP Login: Audit Logging & Error Logging

Custom OCR backend · changes of 2026-11-01 · for applying to the office development environment

1. Summary

Two changes were made to the Active Directory (LDAP) login:

1. **Login attempt audit log.** Every LDAP login attempt (success, failure or error) writes one line to a dedicated rotating log file `ldap_login.log`, with the result, user name, client IP and reason. Passwords are never logged.
2. **Detailed error logging.** A login that previously failed with a bare HTTP 500 now logs the exception type, message and full traceback. LDAP connection, bind and search failures are logged with Active Directory's own reason code. The 500 response names the exception type.

No new Python packages, configuration settings or database changes are required.

File	Function / item	Status	Change
`backend/app/core/logging_config.py`	`configure_logging()`	Modified	Adds the rotating `ldap_login.log` handler; also attaches it to the `ocr.ldap` logger.
	`LDAP_LOGIN_LOGGER`, `LDAP_LOGIN_LOG_FILE`	New	Logger name and file name constants.
	`ldap_login_logger()`	New	Returns the audit logger.
`backend/app/api/v1/auth.py`	imports, `_CONTROL_CHARS`	New	Import of `ldap_login_logger`; regex used to clean user-supplied text.
	`_log_ldap_login()`	New	Writes one audit line (and the traceback, when given an exception).
	`login()`	Modified	Now takes `request`; wraps `_login()` and logs any unexpected exception in full, returning a descriptive 500.
	`_login()`	New	The previous body of `login()`, plus an audit log call for each LDAP outcome.
`backend/app/services/ldap_service.py`	`_server()`	Modified	Debug line with server, host and timeout.
	`_bind()`	Modified	Logs connection errors with traceback; logs rejected binds with AD's reason code.

	`_safe_unbind()`	New	Unbind that never hides the original error.
	`_describe()`	New	Formats ldap3's `conn.result` for the log.
	`_search()`	Modified	Debug line with base DN and filter; error lines on failure.
	`find_user()`	Modified	Warning when 0 or more than 1 account matches; debug line with resolved DN.
	`authenticate()`	Modified	Warning on empty password; info line when the password is verified.
	`search_users()`, `directory_domains()`	Modified	`conn.unbind()` replaced with `_safe_unbind(conn)`.

2. The log file

Location	`<LOGS_PATH>/ldap_login.log`; when `LOGS_PATH` is empty: `<STORAGE_PATH>/logs/ldap_login.log` (default `backend/storage/logs/ldap_login.log`; in Docker under the storage volume).
Rotation	10 MB per file, 10 backups (`ldap_login.log.1` … `.10`), UTF-8.
Contents	One audit line per LDAP login attempt (logger `ocr.auth.ldap_login`, file only), plus INFO/WARNING/ERROR diagnostics from the LDAP service (logger `ocr.ldap`, also shown on the console).
Not included	Local accounts (`administrator` / first-run admin) and Windows SSO sign-ins. Passwords are never written.
Security	Control characters (e.g. new lines) in the user name and reason are replaced, so a crafted user name cannot forge extra log lines. User name limited to 128 characters, reason to 500.

Audit line format

```
<timestamp> result=<SUCCESS|FAILED|ERROR> user=<login name> ip=<client IP> reason=<reason or ->
```

Situation	result	reason	HTTP
Password accepted by AD, user enabled	SUCCESS	`-`	200
User not registered in the application	FAILED	not registered in the application	401
Local account entered as `DOMAIN\name`	FAILED	local account, AD-style login refused	401
AD rejected the credentials	FAILED	AD rejected credentials (wrong password, disabled or removed)	401
User disabled in the application	FAILED	disabled in the application	403
AD unreachable / misconfigured	ERROR	Active Directory unavailable: <details>	503

Any unexpected exception	ERROR	unexpected <module.ExceptionType>: <message> + full traceback	500

Sample log (from the verification run)

```
2026-11-01 15:30:30,843 result=SUCCESS user=jdoe ip=10.1.2.3 reason=-
2026-11-01 15:34:53,820 LDAP - bind rejected for CN=John Doe,DC=dev: result=49 description=invalidCredentials
message=80090308: LdapErr: DSID-0C09044E, comment: AcceptSecurityContext error, data 52e, v4563
2026-11-01 15:34:53,821 result=FAILED user=jdoe ip=10.1.2.3 reason=AD rejected credentials (wrong password,
disabled or removed)
2026-11-01 15:34:53,837 LDAP - cannot reach ldap://dev.team.ae while binding as svc:
ldap3.core.exceptions.LDAPSocketOpenError: socket connection error while opening: timed out
Traceback (most recent call last):
  File "...\app\services\ldap_service.py", line 77, in _bind
    conn.open()
  ...
ldap3.core.exceptions.LDAPSocketOpenError: socket connection error while opening: timed out
2026-11-01 15:34:53,837 result=ERROR user=jdoe ip=10.1.2.3 reason=Active Directory unavailable: Active
Directory is unreachable (ldap://dev.team.ae): socket connection error while opening: timed out
2026-11-01 15:34:53,509 result=ERROR user=jdoe ip=10.1.2.3 reason=unexpected builtins.KeyError:
'defaultNamingContext'
Traceback (most recent call last):
  File "...\app\api\v1\auth.py", line 146, in login
  ...
KeyError: 'defaultNamingContext'
```

Active Directory reason codes

A "bind rejected" line contains AD's message; the code after `data` tells why the login failed.

Code	Meaning	Code	Meaning
`52e`	Wrong password	`533`	Account disabled
`525`	User not found	`701`	Account expired
`530`	Not permitted to log on at this time	`773`	User must change password
`531`	Not permitted to log on from this workstation	`775`	Account locked out
`532`	Password expired		

3. Changes by file

3.1 `backend/app/core/logging_config.py`

Modified `configure_logging()`

- New block after the job log handler: logger `ocr.auth.ldap_login` (level INFO, `propagate = False` so audit lines go to the file only).
- New `RotatingFileHandler` for `<logs_path>/ldap_login.log` (10 MB, 10 backups, UTF-8), format `%(asctime)s %(message)s`, level INFO.
- The same handler is attached to logger `ocr.ldap`, so LDAP diagnostics also reach the file (one handler for one file, so rotation works on Windows).

from line 12

```
def configure_logging() -> None:
    global _configured
```

```
    if _configured:
        return
    level = logging.DEBUG if settings.is_development else logging.INFO
    logging.basicConfig(level=level, format="%(asctime)s %(levelname)s %(name)s: %(message)s")

    os.makedirs(settings.logs_path, exist_ok=True)
    job_logger = logging.getLogger("ocr.jobs")
    job_logger.setLevel(logging.INFO)
    handler = RotatingFileHandler(
        os.path.join(settings.logs_path, "ocr_jobs.log"),
        maxBytes=10 * 1024 * 1024,
        backupCount=5,
        encoding="utf-8",
    )
    handler.setFormatter(logging.Formatter("%(asctime)s %(message)s"))
    job_logger.addHandler(handler)

    # Audit trail of every Active Directory (LDAP) login attempt. File only, never passwords.
    login_logger = logging.getLogger(LDAP_LOGIN_LOGGER)
    login_logger.setLevel(logging.INFO)
    login_logger.propagate = False
    login_handler = RotatingFileHandler(
        os.path.join(settings.logs_path, LDAP_LOGIN_LOG_FILE),
        maxBytes=10 * 1024 * 1024,
        backupCount=10,
        encoding="utf-8",
    )
    login_handler.setFormatter(logging.Formatter("%(asctime)s %(message)s"))
    login_handler.setLevel(logging.INFO)
    login_logger.addHandler(login_handler)
    # LDAP diagnostics (bind rejections with AD's reason code, connection / search
    # errors with tracebacks) go to the same file as well as the console.
    logging.getLogger("ocr.ldap").addHandler(login_handler)
    _configured = True
```

New constants

- Module-level names used above (defined below the function; they are resolved when it runs).

```
LDAP_LOGIN_LOGGER = "ocr.auth.ldap_login"
LDAP_LOGIN_LOG_FILE = "ldap_login.log"
```

New `ldap_login_logger()`

- Accessor for the audit logger, imported by `auth.py`.

from line 58

```
def ldap_login_logger() -> logging.Logger:
    return logging.getLogger(LDAP_LOGIN_LOGGER)
```

3.2 backend/app/api/v1/auth.py

New import and constant

- Import of the audit logger accessor (with the other `app.core` imports).
- `_CONTROL_CHARS`: control characters replaced in logged user input.

```
from app.core.logging_config import ldap_login_logger
_CONTROL_CHARS = re.compile(r"[\x00-\x1f\x7f]")
```

New `_log_ldap_login()`

- Writes one audit line: `result`, cleaned user name (max 128 chars), client IP (`request.client.host`), cleaned reason (max 500 chars).
- With `exc`: logged at ERROR with the full traceback under the line; otherwise INFO.

from line 126

```
def _log_ldap_login(
    request: Request, username: str, result: str, reason: str = "", exc: Optional[BaseException] = None
) -> None:
    """One line per AD login attempt in <LOGS_PATH>/ldap_login.log. Never logs the password.

    When `exc` is given the full traceback is written below the line.
    """
    # The user name is attacker-controlled: strip control characters so it can't forge log lines.
    name = _CONTROL_CHARS.sub("?", username or "")[:128] or "-"
    reason = _CONTROL_CHARS.sub(" ", reason or "")[:500] or "-"
    ip = request.client.host if request.client else "-"
    level = logging.ERROR if exc is not None else logging.INFO
    ldap_login_logger().log(
        level, "result=%s user=%s ip=%s reason=%s", result, name, ip, reason, exc_info=exc
    )
```

Modified `login()`

- Signature gains `request: Request` (needed for the client IP).
- Body replaced by a wrapper around `_login()`. `HTTPException` (401/403/503) passes through unchanged.
- Any other exception: `logger.exception(...)` (console, with traceback) and `_log_ldap_login(..., "ERROR", ..., exc)` (file, with traceback); the client receives HTTP 500 *Login failed because of a server error (<ExceptionType>). See ldap_login.log for details.*

from line 143

```
@router.post("/login")
def login(payload: LoginRequest, request: Request, db: Session = Depends(get_db)) -> dict:
    try:
        return _login(payload, request, db)
    except HTTPException:
        raise  # expected outcomes (401/403/503), already logged
    except Exception as exc:
        # Unexpected failure (would otherwise be a bare 500): log the full traceback
        # to the console and to ldap_login.log.
        logger.exception("AUTH - unexpected error during login of %r", payload.username)
        _log_ldap_login(
            request, payload.username, "ERROR", f"unexpected {type(exc).__module__}.{type(exc).__name__}:
{exc}", exc
        )
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Login failed because of a server error ({type(exc).__name__}). See ldap_login.log for
details.",
        ) from exc
```

New `_login()`

- Contains the previous body of `login()`; the local-account branch is unchanged.
- In the LDAP branch one `_log_ldap_login()` call was added before each outcome: not registered (FAILED), local account via AD-style name (FAILED), `LdapError` (ERROR), AD rejected credentials (FAILED), disabled (FAILED), success (SUCCESS).

from line 162

```
def _login(payload: LoginRequest, request: Request, db: Session) -> dict:
    # Local accounts are verified here, never against AD.
    user = find_user(db, payload.username) if payload.username.strip() else None
    if user is not None and is_local(user):
        if not verify_password(payload.password, user.hashed_password):
            logger.warning("AUTH - local login failed for %s", user.username)
            raise _invalid()
        if not user.enabled:
            logger.warning("AUTH - login refused, %s is disabled", user.username)
            raise _disabled()
        return _session(user)

    # Everyone else must exist in our DB (added by an admin) AND in AD.
    username = ldap_service.normalize_username(payload.username)
    user = find_user(db, username) if username else None
    if user is None:
        logger.warning("AUTH - login refused, %s is not registered in the application", username)
        _log_ldap_login(request, username or payload.username, "FAILED", "not registered in the
application")
        raise _invalid()
    if is_local(user):
        # e.g. "DEV\administrator": a local account must never be unlocked by an AD password.
        logger.warning("AUTH - %s is a local account; AD-style login refused", user.username)
        _log_ldap_login(request, payload.username, "FAILED", "local account, AD-style login refused")
        raise _invalid()
    try:
        ad_user = ldap_service.authenticate(username, payload.password)
    except ldap_service.LdapError as exc:
        logger.error("AUTH - AD unavailable during login of %s: %s", username, exc)
        _log_ldap_login(request, username, "ERROR", f"Active Directory unavailable: {exc}")
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="Active Directory is unavailable. Try again later or contact the administrator.",
        ) from exc
    if ad_user is None:
        logger.warning("AUTH - AD rejected login for %s (wrong password, disabled or removed)", username)
        _log_ldap_login(request, username, "FAILED", "AD rejected credentials (wrong password, disabled or
removed)")
        raise _invalid()
    if not user.enabled:
        logger.warning("AUTH - login refused, %s is disabled in the application", user.username)
        _log_ldap_login(request, user.username, "FAILED", "disabled in the application")
        raise _disabled()
    if ad_user.display_name and ad_user.display_name != user.display_name:
        user.display_name = ad_user.display_name
        db.commit()
    _log_ldap_login(request, user.username, "SUCCESS")
    return _session(user)
```

3.3 backend/app/services/ldap_service.py

Modified `_server()`

- Added a DEBUG line with the server URL, host and connect timeout.

from line 58

```
def _server() -> Server:
    host = urlparse(settings.LDAP_SERVER).hostname or settings.LDAP_SERVER
    # The offline guard blocks public IPs; the DC is an explicitly trusted host.
    network_guard.allow_host(host)
    logger.debug("LDAP - server %s (host %s, timeout %ss)", settings.LDAP_SERVER, host,
settings.LDAP_TIMEOUT_S)
    return Server(settings.LDAP_SERVER, get_info=DSA, connect_timeout=settings.LDAP_TIMEOUT_S)
```

Modified **`_bind()`**

- Successful bind: DEBUG line.
- `LDAPException` / `OSError` (server unreachable, TLS, timeout): ERROR line with the bind user, exception module/type/message and traceback, then `LdapError` as before.
- Rejected bind: WARNING line with AD's `result`, `description` and `message` (contains the `data XXX` reason code).
- `conn.unbind()` replaced by `_safe_unbind(conn)`.

from line 66

```
def _bind(server: Server, user: str, password: str) -> Optional[Connection]:
    """Return a bound connection, None on bad credentials, LdapError when unreachable."""
    conn = Connection(
        server,
        user=user,
        password=password,
        authentication=SIMPLE,
        receive_timeout=settings.LDAP_TIMEOUT_S,
        raise_exceptions=False,
    )
    try:
        conn.open()
        if settings.LDAP_USE_STARTTLS:
            conn.start_tls()
        if conn.bind():
            logger.debug("LDAP - bind OK as %s", user)
            return conn
    except (LDAPException, OSError) as exc:
        logger.error(
            "LDAP - cannot reach %s while binding as %s: %s.%s: %s",
            settings.LDAP_SERVER, user, type(exc).__module__, type(exc).__name__, exc, exc_info=True,
        )
        _safe_unbind(conn)
        raise LdapError(f"Active Directory is unreachable ({settings.LDAP_SERVER}): {exc}") from exc
    # AD's diagnostic text says why, e.g. "AcceptSecurityContext error, data 52e" (52e = wrong
    # password, 525 = no such user, 530/531 = logon time/workstation, 532 = password expired,
    # 533 = disabled, 701 = account expired, 773 = must change password, 775 = locked out).
    logger.warning("LDAP - bind rejected for %s: %s", user, _describe(conn.result))
    _safe_unbind(conn)
    return None
```

New **`_safe_unbind()`**

- Unbinds and swallows (DEBUG-logs) any error, so closing a broken connection cannot replace the real error.

from line 98

```
def _safe_unbind(conn: Connection) -> None:
    try:
        conn.unbind()
    except Exception:  # closing a broken socket must not hide the real error
        logger.debug("LDAP - unbind failed", exc_info=True)
```

New **`_describe()`**

- Formats ldap3's `conn.result` as `result=… description=… message=…`.

from line 105

```
def _describe(result: Optional[dict]) -> str:
    result = result or {}
    return (
        f"result={result.get('result')} description={result.get('description')} "
```

```
        f"message={(result.get('message') or '').strip() or '-'}"
    )
```

Modified `_search()`

- DEBUG line with the base DN and filter before searching.
- Exception during search: ERROR line with type, message and traceback.
- Unsuccessful result code: ERROR line with `_describe(conn.result)`.

from line 159

```
def _search(conn: Connection, condition: str, limit: int) -> list[LdapUser]:
    logger.debug("LDAP - search base=%s filter=%s", _base_dn(conn), condition)
    try:
        ok = conn.search(
            _base_dn(conn),
            f"(&{_USER_FILTER}{condition})",
            search_scope=SUBTREE,
            attributes=_ATTRIBUTES,
            size_limit=limit,
        )
    except (LDAPException, OSError) as exc:
        logger.error("LDAP - search failed (%s): %s.%s: %s", condition, type(exc).__module__,
type(exc).__name__, exc,
                     exc_info=True)
        raise LdapError(f"Active Directory search failed: {exc}") from exc
    # sizeLimitExceeded still returns the first `limit` entries — that's fine.
    if not ok and conn.result.get("result") not in (0, 4):
        logger.error("LDAP - search failed (%s): %s", condition, _describe(conn.result))
        raise LdapError(f"Active Directory search failed: {conn.result.get('description')}")
    users = []
    for entry in conn.response or []:
        if entry.get("type") != "searchResEntry":
            continue  # skip referrals
        attrs = entry["attributes"]
        username = _first(attrs, "sAMAccountName")
        if not username:
            continue
        users.append(
            LdapUser(
                username=username,
                dn=entry["dn"],
                display_name=_first(attrs, "displayName"),
                email=_first(attrs, "mail"),
                upn=_first(attrs, "userPrincipalName"),
                department=_first(attrs, "department"),
                title=_first(attrs, "title"),
            )
        )
    return users
```

Modified `find_user()`

- `_safe_unbind(conn)` in `finally`.
- WARNING when the number of enabled matching accounts is not exactly 1 (not found, disabled, or duplicate); DEBUG line with the resolved DN.

from line 215

```
def find_user(username: str) -> Optional[LdapUser]:
    """Exact lookup by login name (sAMAccountName). None = not in AD or disabled."""
    name = escape_filter_chars(normalize_username(username))
    if not name:
        return None
    conn = _service_connection()
    try:
        found = _search(conn, f"(sAMAccountName={name})", 2)
```

```
    finally:
        _safe_unbind(conn)
    if len(found) != 1:
        logger.warning("LDAP - %s: %d enabled AD account(s) found (expected 1)", name, len(found))
        return None
    logger.debug("LDAP - %s resolved to %s", name, found[0].dn)
    return found[0]
```

Modified `authenticate()`

- WARNING when an empty password is refused.
- `_safe_unbind(conn)` after the user bind; INFO line when the password is verified.

from line 232

```
def authenticate(username: str, password: str) -> Optional[LdapUser]:
    """Verify the password against AD. None = unknown/disabled user or wrong password."""
    # An empty password would be an "unauthenticated bind" that AD reports as
    # successful — never let it through.
    if not password:
        logger.warning("LDAP - login of %s refused: empty password", username)
        return None
    user = find_user(username)
    if user is None:
        return None
    conn = _bind(_server(), user.dn, password)
    if conn is None:
        return None
    _safe_unbind(conn)
    logger.info("LDAP - password verified for %s", user.username)
    return user
```

Modified `search_users()`, `directory_domains()`

- Only change: `conn.unbind()` in the `finally` block replaced by `_safe_unbind(conn)`.

4. Applying in the office environment

1. Copy (or apply the changes above to) the three files: `backend/app/core/logging_config.py`, `backend/app/api/v1/auth.py`, `backend/app/services/ldap_service.py`.
2. Restart the backend. No packages, environment variables or database changes are needed.
3. Log in with an AD user and check `<LOGS_PATH>/ldap_login.log` (default `backend/storage/logs/ldap_login.log`).
4. For an HTTP 500, find the `result=ERROR` line and the traceback under it; the same traceback is on the console.

Client IP behind a reverse proxy: if the backend runs behind nginx or another proxy, the `ip=` field shows the proxy's address unless uvicorn is started with `--proxy-headers` (and `--forwarded-allow-ips` set to the proxy address).

5. Verification

Tested with FastAPI `TestClient`, a temporary database and a mocked ldap3 connection:

Test	Response	Log

Correct password	200	`result=SUCCESS`
Wrong password (bind rejected, `data 52e`)	401	"bind rejected" line with AD message + `result=FAILED`
Unregistered user	401	`result=FAILED … not registered in the application`
User name containing a new line	401	Single line; new line replaced by `?`
AD socket timeout	503	"cannot reach" line with `LDAPSocketOpenError` traceback + `result=ERROR`
Forced `KeyError` inside AD code	500 naming `KeyError`	`result=ERROR … unexpected builtins.KeyError` + full traceback